SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: COMPUTER ARCHITECTURE COURSE CODE: CSA1221

COURSE OUTCOME COVERED

CO1: Analyse the functional units of a computer system including CPU, memory, bus structures, and I/O components by examining instruction execution cycles, addressing modes, and performance metrics such as CPI, clock cycles, and benchmarking (BL4)

Assessment Tool Weightage: 40%

QUESTIONS:5 Total Marks:25

ANNEXURE A

APPLICATION BASED QUESTIONS

Code of Conduct:

I K.NISHALINI (192521301) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

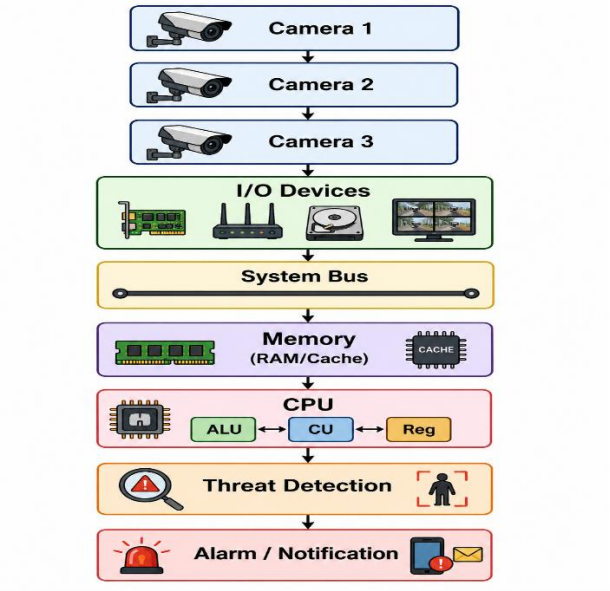
ANNEXURE A

1. A smart city surveillance system processes continuous video feeds from multiple cameras, where large volumes of data are transferred between I/O devices, memory, and CPU. During peak hours, the system experiences lag and delayed response in threat detection. Analyze how the interaction between CPU, memory, and I/O contributes to this issue, and explain how different bus structures (single-bus vs multi-bus) and improvements in instruction execution cycle can enhance system performance.

1. Introduction

A smart city surveillance system continuously collects video data from cameras installed at roads, public places, airports, and railway stations. The CPU processes these video streams by receiving data from I/O devices through memory. During peak hours, multiple cameras send large amounts of data simultaneously, causing delays in processing and threat detection.

2. System Block Diagram



3. Interaction between CPU, Memory and I/O

CPU

- Executes surveillance software.
- Detects suspicious activities.
- Controls data processing.

Memory

- Temporarily stores video frames.
- Supplies data to CPU quickly.
- Cache memory speeds up processing.

I/O Devices

- CCTV Cameras provide input.
- Storage devices save recordings.
- Network devices transmit video.

Working

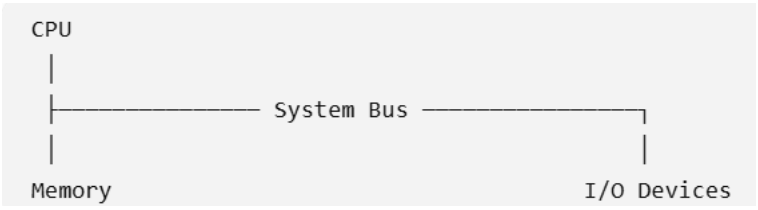
1. Cameras capture live video.
2. Video is transferred through I/O.
3. Data is stored in RAM.
4. CPU fetches instructions.
5. CPU processes video.
6. Threat is detected.
7. Alert is generated.

4. Why Lag Occurs During Peak Hours

During busy hours,

- Hundreds of cameras send video simultaneously.
- CPU receives too many requests.
- Memory becomes overloaded.
- System bus becomes busy.
- CPU waits for data.
- Processing speed decreases.
- Threat detection becomes slow.

5. Single Bus Structure



Advantages

- Simple design
- Low cost

- Easy maintenance

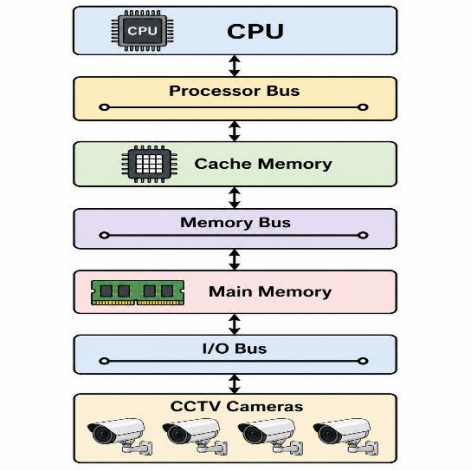
Disadvantages

- Only one transfer at a time
- Bus congestion
- CPU waits frequently
- Slow performance

Result

Large surveillance systems suffer from heavy delays.

6. Multi Bus Structure



Advantages

- Multiple transfers occur simultaneously.
- Less congestion.
- Faster communication.
- Better CPU utilization.
- High throughput.

Disadvantages

- Cost is higher.
- Design is more complex.

Result

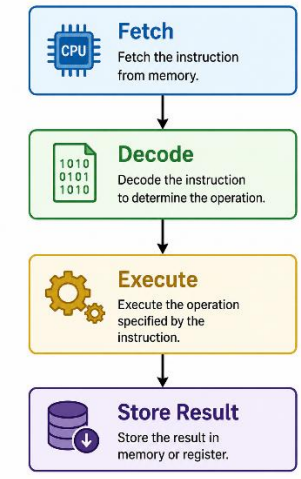
Suitable for smart city surveillance.

7. Comparison

Feature	Single Bus	Multi Bus
Number of buses	One	Multiple
Speed	Slow	Fast
Cost	Low	High
CPU Waiting	High	Low
Performance	Low	High
Bus Traffic	Heavy	Reduced

8. Instruction Execution Cycle

The CPU executes every instruction in four stages.



Fetch : Instruction is fetched from memory.

Decode : Control Unit understands the instruction.

Execute :ALU performs the operation.

Store : Result is stored in memory.

9. Improvements in Instruction Execution Cycle

1. Pipelining

```
Instruction 1 → Fetch → Decode → Execute

Instruction 2       Fetch → Decode → Execute

Instruction 3              Fetch → Decode → Execute
```

- Executes multiple instructions simultaneously.
- Improves throughput.
- Reduces delay.

2. Cache Memory

- Frequently used data stored near CPU.
- Reduces memory access time.
- Faster execution.

3. Branch Prediction

- Predicts next instruction.
- Reduces waiting time.
- Improves CPU efficiency.

4. DMA (Direct Memory Access)

```
Camera
   |
DMA Controller
   |
Memory
```

- Data transfers directly between I/O and memory.
- CPU is free for processing.
- Faster data transfer.

5. Multi-core Processor

- Multiple cores process different camera feeds.
- Better parallel processing.
- Faster threat detection.

10. Solutions to Improve Performance

- Use multi-bus architecture.
- Increase RAM capacity.
- Use larger cache memory.
- Implement DMA.
- Use pipelining.
- Install multi-core CPUs.
- Upgrade to high-speed SSD storage.
- Optimize surveillance software algorithms.

11. Real-Time Example

A city has **500 CCTV cameras**.

- During festivals, all cameras transmit HD video.
- In a **single-bus system**, the CPU waits because only one device can use the bus at a time, causing delayed threat detection.
- In a **multi-bus system**, video transfer, memory access, and CPU processing occur simultaneously, enabling faster detection and alerts.

12. Advantages of Improved Architecture

- Faster video processing
- Reduced CPU waiting time
- Higher system throughput
- Better real-time monitoring
- Accurate threat detection
- Reduced lag
- Improved public safety

13. Conclusion

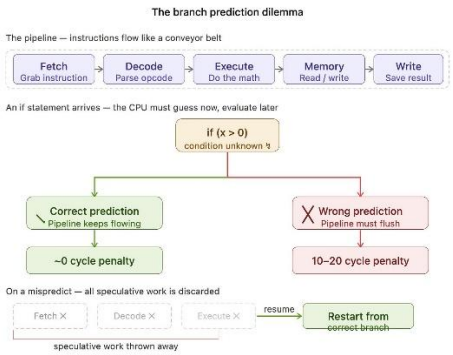
The performance of a smart city surveillance system depends on efficient coordination between the CPU, memory, and I/O devices. During peak hours, heavy data traffic causes bus congestion and increases CPU waiting time. A multi-bus architecture, combined with pipelining, cache memory, DMA, branch prediction, and multi-core processors, significantly improves system performance, reduces lag, and enables faster real-time threat detection, making it ideal for modern surveillance applications.

2. A real-time stock trading application running on a processor executes millions of instructions per second, but users report delays in transaction processing during high load. The system has a high CPI due to frequent memory access and branch instructions. Examine how the fetch–decode– execute cycle impacts execution time in this scenario, and propose methods to reduce CPI and improve overall CPU performance using suitable architectural enhancements.

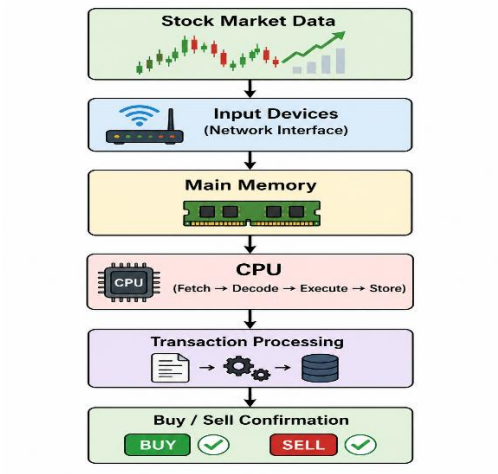
1. Introduction

A real-time stock trading application processes millions of buy and sell transactions every second. The CPU executes numerous instructions to process market data, calculate prices, and confirm transactions. During high trading volume, the system experiences delays because of a high CPI (Cycles Per Instruction) caused by frequent memory accesses and branch instructions.

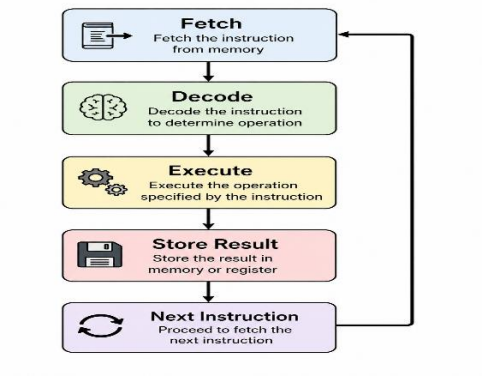
2. System Flow Diagram



3. Working of the System



4. Fetch–Decode–Execute Cycle



Fetch

- CPU fetches instruction from memory.

Decode

- Control Unit decodes the instruction.

Execute

- ALU performs calculations or comparisons.

Store

- Result is written back to memory or registers.

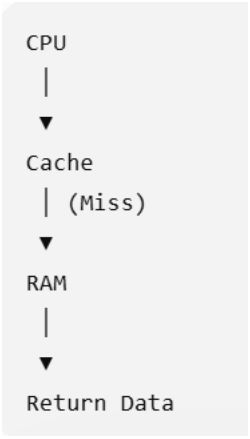
5. How the Cycle Affects Execution Time

During heavy stock trading,

- Millions of instructions are fetched.
- CPU repeatedly accesses memory.
- Memory access takes longer than register access.
- Branch instructions interrupt the instruction flow.
- CPU pipeline becomes stalled.
- More clock cycles are needed.
- CPI increases.
- Transaction processing becomes slower.

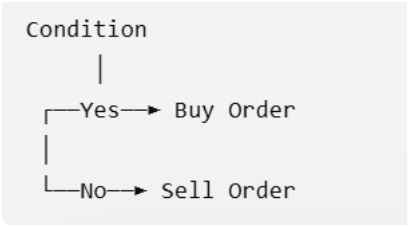
6. Causes of High CPI

a) Frequent Memory Access



- Cache misses increase memory access time.
- CPU waits for RAM.

b) Branch Instructions



CPU waits until the branch decision is completed.

- Incorrect prediction flushes the pipeline.

7. CPI Formula

$$\text{CPU Time} = \text{Instruction Count} \times \text{CPI} \times \text{Clock Cycle Time}$$

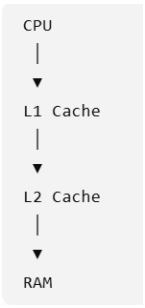
Where:

- Instruction Count = Total instructions executed
- CPI = Cycles per instruction
- Clock Cycle Time = Time for one CPU cycle

Higher CPI results in increased execution time.

8. Architectural Enhancements to Reduce CPI

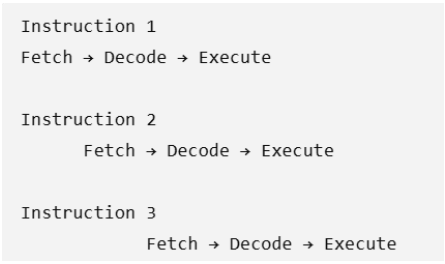
1. Cache Memory



Advantages

- Faster memory access
- Reduced cache misses
- Lower CPI

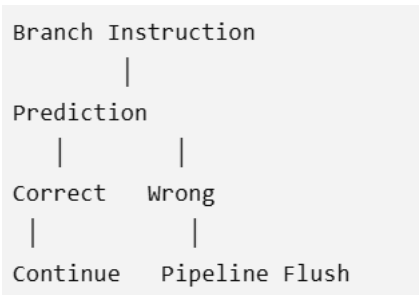
2. Instruction Pipelining



Advantages

- Multiple instructions execute simultaneously.
- Improves throughput.
- Reduces execution time.

3. Branch Prediction

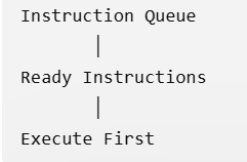


Advantages

- Predicts the next instruction.

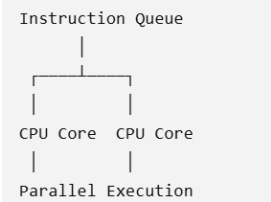
- Reduces branch delay.
- Improves CPU efficiency.

4. Out-of-Order Execution



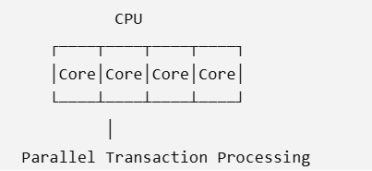
- Executes independent instructions while waiting for memory.
- Reduces idle CPU time.

5. Superscalar Processor



- Executes multiple instructions in one clock cycle.
- Improves overall performance.

6. Multi-Core Processor



- Different cores process different transactions simultaneously.
- Reduces system delay.

9. Comparison of Improvements

Technique	Benefit
Cache Memory	Faster memory access
Pipelining	Executes multiple instructions simultaneously
Branch Prediction	Reduces branch delay
Out-of-Order Execution	Keeps CPU busy
Superscalar Processor	Multiple instructions per cycle
Multi-Core Processor	Parallel transaction processing

10. Real-Time Example

A stock exchange receives 2 million buy/sell requests per second.

Without optimization:

- High memory access delay
- High CPI
- Slow transaction processing
- Delayed confirmations

With optimization:

- Cache memory reduces memory access time.
- Branch prediction minimizes pipeline stalls.
- Pipelining and multi-core CPUs process multiple transactions simultaneously.
- CPI decreases and transaction speed improves.

11. Advantages

- Reduced CPI
- Faster transaction processing
- Better CPU utilization
- Higher throughput
- Lower latency
- Improved customer experience
- Better real-time trading performance

12. Conclusion

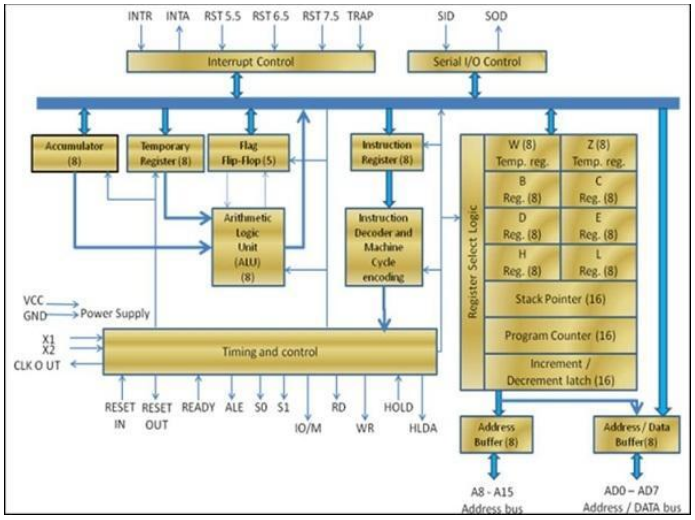
In a real-time stock trading application, the Fetch–Decode–Execute cycle directly influences execution time. Frequent memory accesses and branch instructions increase the Cycles Per Instruction (CPI), causing transaction delays. By using cache memory, pipelining, branch prediction, out-of-order execution, superscalar architecture, and multi-core processors, the CPU can reduce CPI, improve throughput, and deliver faster and more reliable transaction processing during high-load conditions.

3. An embedded system based on an 8085 microprocessor is used in an industrial temperature control unit, where sensors provide input and actuators respond accordingly. However, incorrect temperature readings are occasionally processed due to improper use of addressing modes in the program. Discuss how different addressing modes influence data access, identify possible causes of such errors, and suggest how the instruction set of 8085 can be effectively used to ensure accurate and reliable operation.

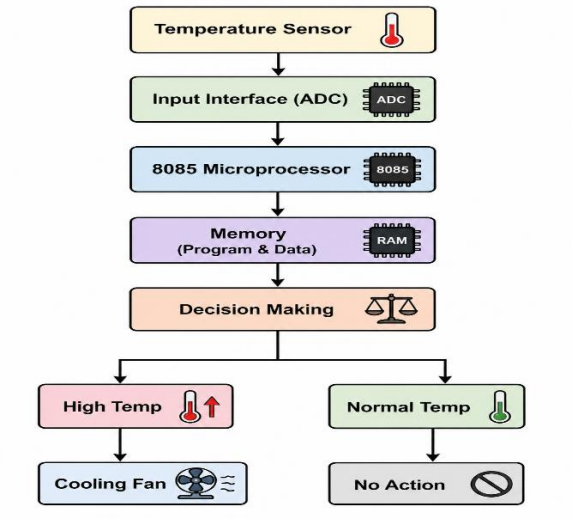
1. Introduction

An embedded system is a computer system designed to perform a specific task. In an industrial temperature control unit, the 8085 microprocessor reads temperature data from sensors, processes it, and controls actuators such as heaters or cooling fans. Incorrect use of addressing modes can cause the CPU to access wrong memory locations, resulting in incorrect temperature readings and unreliable system operation.

2. Embedded Temperature Control System



3. Working Flowchart



4. Role of 8085 Microprocessor

- Reads data from the temperature sensor.
- Stores sensor values in memory.
- Compares the temperature with a preset value.
- Controls actuators like heaters or cooling fans.
- Repeats the process continuously.

5. Addressing Modes in 8085

a) Immediate Addressing Mode

MVI A, 30H

- Data is directly available in the instruction.
- Faster execution.
- Used for constant values.

Application

Store the temperature limit.

b) Register Addressing Mode

MOV A, B

- Data is transferred between registers.
- No memory access required.
- Fast execution.

Application

Move sensor data between registers.

c) Direct Addressing Mode

LDA 2500H

- CPU reads data directly from memory location 2500H.

Application

Read stored temperature value.

d) Register Indirect Addressing Mode

MOV A, M

- HL register pair stores the memory address.
- CPU accesses data indirectly.

Application

Read multiple sensor values stored in memory.

e) Implied Addressing Mode

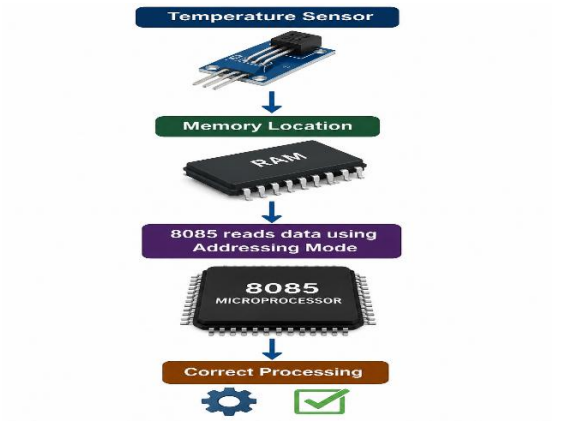
CMA

- Operand is implied.
- No address is specified.

Application

Complement data when required.

6. How Addressing Modes Influence Data Access



Different addressing modes determine where the CPU gets the data.

Example:

If the wrong addressing mode is used,

- CPU reads incorrect memory.
- Wrong temperature is processed.
- Incorrect control action occurs.

7. Possible Causes of Incorrect Temperature Readings

1. Wrong Memory Address

Example:

Instead of

LDA 2500H

Program uses

LDA 2600H

Wrong data is read.

2. Incorrect Register Selection

Example

MOV A,C

instead of

MOV A,B

Temperature value becomes incorrect.

3. HL Register Not Initialized

MOV A,M

If HL points to the wrong location,

CPU reads garbage data.

4. Sensor Noise :Electrical noise changes sensor readings.

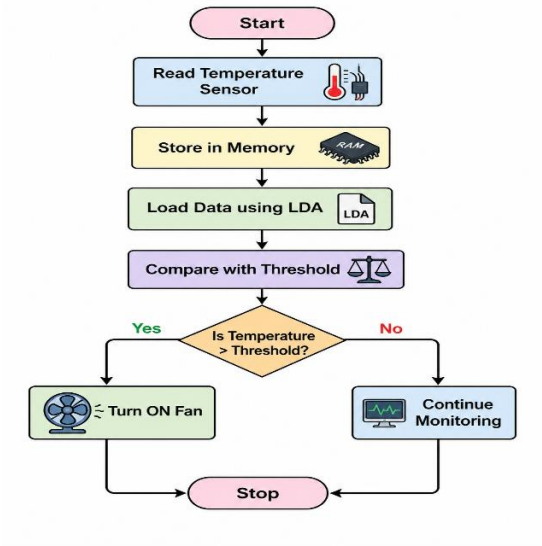
5. Memory Corruption :Temperature value stored incorrectly.

6. Programming Errors :Wrong instruction sequence.

8. 8085 Instruction Set Used

Instruction	Purpose
LDA	Load temperature from memory
STA	Store processed value
MOV	Transfer data
MVI	Load immediate value
CMP	Compare temperature
JZ	Jump if equal
JC	Jump if carry
JMP	Unconditional jump
IN	Read sensor data
OUT	Send signal to actuator

9. Flowchart of Program Execution



10. Methods to Ensure Accurate Operation

✓ Use Correct Addressing Mode

Always use the correct addressing mode for accessing sensor data.

✓ Initialize Registers Properly

Set HL, BC, and DE correctly before execution.

✓ Validate Sensor Data

Check whether the sensor value is within the valid temperature range.

✓ Use Proper Instructions

Use instructions like

- LDA
- MOV
- CMP
- JC
- JZ
- STA

correctly.

✓ Error Checking

Ignore invalid sensor values.

✓ Regular Calibration

Calibrate sensors periodically.

11. Real-Time Example

An industrial boiler must maintain 80°C.

- Sensor sends temperature data to the 8085.
- CPU reads the value using LDA.
- CPU compares the value using CMP.
- If temperature exceeds 80°C, the cooling fan is switched ON using the OUT instruction.
- Correct addressing modes ensure the CPU always reads the correct sensor value and prevents overheating.

12. Advantages

- Accurate temperature measurement
- Reliable industrial control
- Faster data access
- Reduced programming errors
- Efficient memory utilization
- Safe operation
- Improved system performance

13. Conclusion

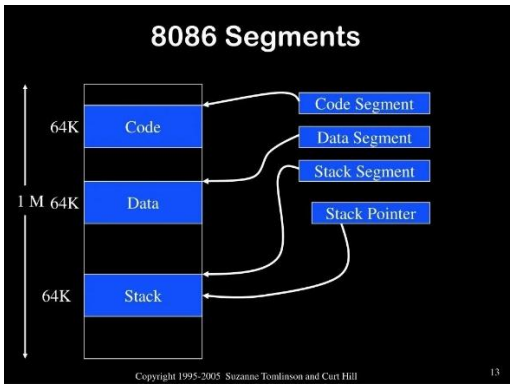
In an 8085-based embedded temperature control system, addressing modes determine how the CPU accesses sensor data. Incorrect addressing modes can lead to wrong memory access, incorrect temperature readings, and improper actuator control. By selecting the correct 8085 addressing modes, initializing registers properly, validating sensor inputs, and using instructions such as LDA, MOV, CMP, JZ, JC, STA, IN, and OUT, the system achieves accurate, reliable, and efficient temperature monitoring and control.

4. A computing system uses an 8086 microprocessor with segmented memory for executing multiple programs simultaneously, but it encounters issues like incorrect data access and stack overflow errors. Evaluate how segmentation (code, data, and stack segments) works in 8086, analyze how improper segment handling can lead to such problems, and explain how instruction execution and addressing modes must be managed to ensure correct program execution.

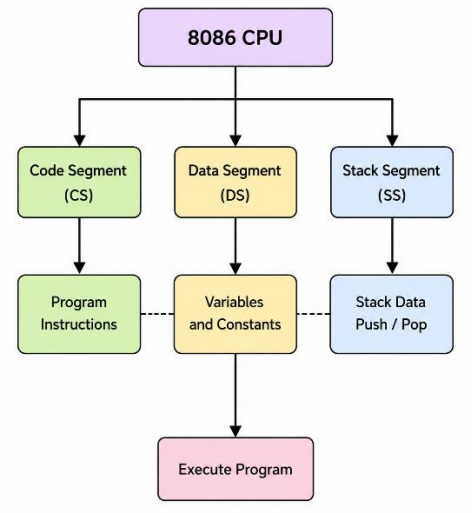
1. Introduction

The 8086 microprocessor uses segmented memory architecture to efficiently utilize its 1 MB memory space. Memory is divided into Code Segment (CS), Data Segment (DS), Stack Segment (SS), and Extra Segment (ES). Proper management of these segments ensures correct execution of multiple programs. Incorrect segment handling can cause wrong data access, stack overflow, and program execution errors.

2. 8086 Segmented Memory Architecture



3. Memory Segmentation Flowchart



4. Segments in 8086

a) Code Segment (CS)

- Stores program instructions.
- Controlled by the CS register.

- CPU fetches instructions from this segment.

Example

MOV AX, BX

ADD AX, CX

b) Data Segment (DS)

- Stores variables and data.
- Controlled by the DS register.
- Used for reading and writing data.

c) Stack Segment (SS)

- Stores temporary data.
- Controlled by the SS register.
- Used by PUSH, POP, CALL, and RET instructions.

d) Extra Segment (ES)

- Used for additional data storage.
- Mainly used in string operations.

5. Physical Address Calculation

The 8086 calculates the physical address using:

Physical Address = Segment Address × 10H + Offset Address

Example

CS = 2000H

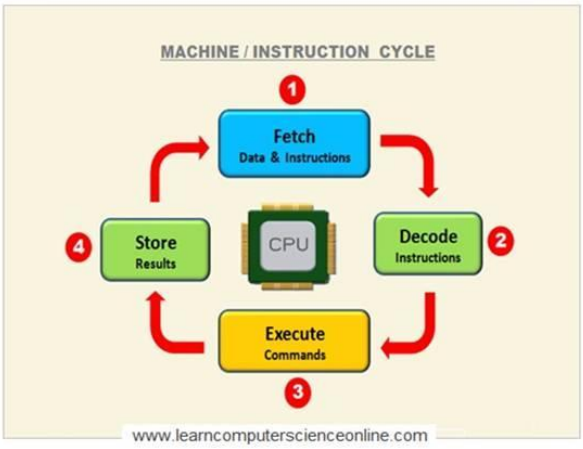
IP = 1200H

Physical Address

= 2000H × 10H + 1200H

= 21200H

6. Instruction Execution Cycle



7. Addressing Modes in 8086

1. Immediate Addressing

MOV AX,1234H

- Constant data is directly stored.

2. Register Addressing

MOV AX,BX

- Data transfer between registers.

3. Direct Addressing

MOV AX,[2500H]

- Data is accessed directly from memory.

4. Register Indirect Addressing

MOV AX,[BX]

- BX register contains memory address.

5. Based Indexed Addressing

MOV AX,[BX+SI]

- Uses base and index registers.
- Efficient for arrays.

8. Problems Due to Improper Segment Handling

a) Incorrect Data Access

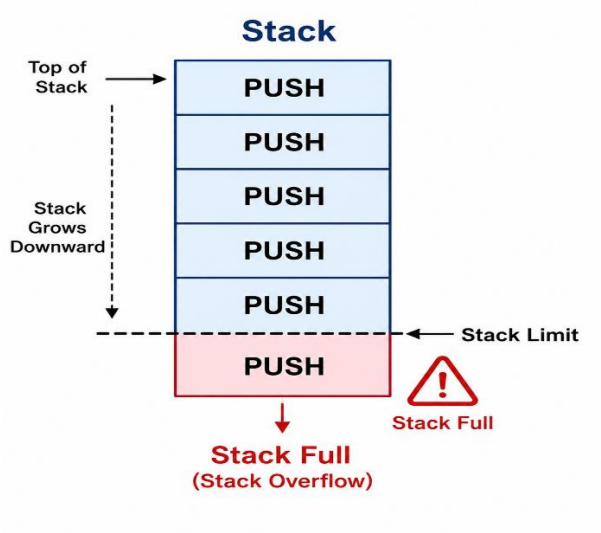
```
Correct

DS → Temperature Data

Incorrect

DS → Wrong Memory
```

Result:

- Wrong variable accessed.
- Incorrect calculations.
- Program failure.

b) Stack Overflow

Result:



- Stack exceeds memory limit.
- Program crashes.
- Incorrect return address.

c) Wrong Segment Register

If CS, DS, or SS contains incorrect values,

- CPU accesses wrong memory.
- Execution becomes unpredictable.

d) Invalid Offset Address

Wrong offset causes access to invalid memory locations.

9. Managing Instruction Execution Correctly

✓ Load Correct Segment Registers

Example

```
MOV AX,DATA
MOV DS,AX
```

✓ Proper Stack Initialization

```
MOV AX,STACK
MOV SS,AX
MOV SP,2000H
```

✓ Use Correct Addressing Modes

Use

- Immediate
- Register
- Direct
- Register Indirect

according to the application.

✓ Prevent Stack Overflow

Balance every

PUSH

with

POP

✓ Use CALL and RET Properly

Ensures correct program return.

10. Real-Time Example

A factory automation system uses an 8086 microprocessor to monitor machines.

- Code Segment (CS): Stores the control program.
- Data Segment (DS): Stores machine temperature and pressure values.
- Stack Segment (SS): Stores temporary values during subroutine execution.

- Extra Segment (ES): Stores communication buffers.

If the DS register points to the wrong memory location, incorrect sensor values are read. If the stack is not properly managed, repeated PUSH operations can cause a stack overflow, leading to program crashes. Correct initialization of CS, DS, SS, and proper use of addressing modes ensure reliable and error-free operation.

11. Advantages of Proper Segmentation

- Efficient memory utilization
- Supports multitasking
- Faster program execution
- Organized memory management
- Reduces memory conflicts
- Improves reliability
- Simplifies debugging

12. Conclusion

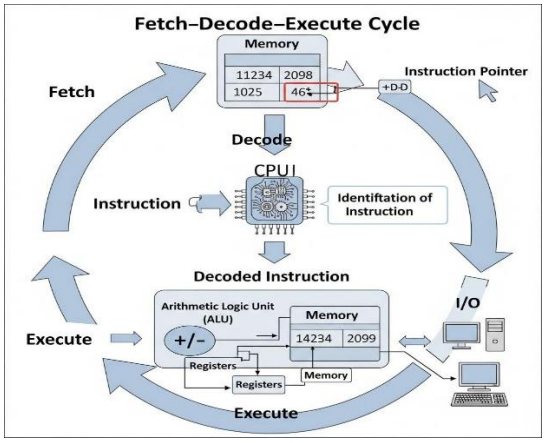
The 8086 segmented memory architecture divides memory into Code, Data, Stack, and Extra Segments, allowing efficient execution of multiple programs. Incorrect segment register values or improper stack handling can cause wrong data access and stack overflow errors. By correctly initializing CS, DS, SS, and ES, using suitable addressing modes, and properly managing the Fetch–Decode–Execute cycle, the system ensures accurate, reliable, and efficient program execution in real-time applications.

5. A data communication system represents packet information in hexadecimal format for ease of debugging and transmission, but a software engineer mistakenly interprets values, leading to incorrect data processing. Analyze the importance of number system conversions between hexadecimal and binary in such systems, explain how errors in conversion can affect instruction execution, and discuss how efficient CPU design and benchmarking can help detect and minimize such issues in real-time systems.

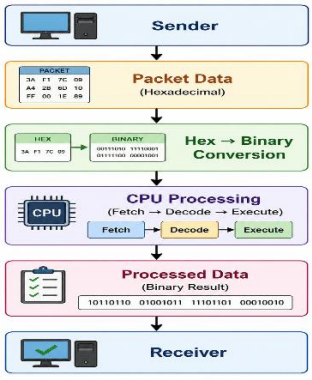
1. Introduction

A data communication system transfers information between computers using data packets. Packet information is commonly represented in hexadecimal (Hex) because it is compact and easy to read. Internally, however, the CPU processes all data in binary. Therefore, accurate conversion between hexadecimal and binary is essential for correct data processing and instruction execution.

2. Data Communication System Block Diagram



3. Working Flowchart



4. Number Systems

Binary (Base 2)

- Digits: 0 and 1
- Used internally by the CPU.

Example

10101100_2

Hexadecimal (Base 16)

- Digits: 0–9 and A–F
- Easier to read and debug.

Example

AC_{16}

5. Hexadecimal to Binary Conversion

Hexadecimal	Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111

Example Conversion

Hexadecimal = $3A_{16}$

3 = 0011

A = 1010

Binary = 00111010_2

6. Importance of Hexadecimal and Binary Conversion

- Makes debugging easier.
- Reduces long binary numbers.
- Simplifies memory addressing.
- Helps in packet analysis.
- Enables correct instruction execution.
- Improves communication accuracy.

7. Errors Due to Incorrect Conversion

Example

Correct:

$2F_{16} = 00101111_2$

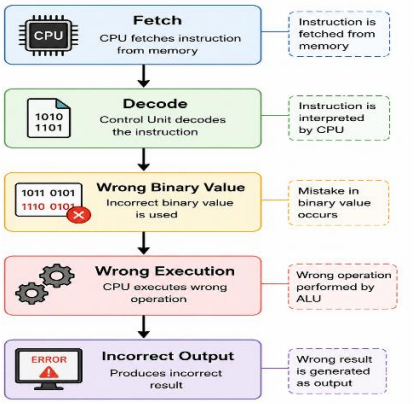
Wrong:

$2F_{16} = 00111111_2$

This incorrect conversion changes the binary value and causes the CPU to process the wrong data.

8. Effect on Instruction Execution

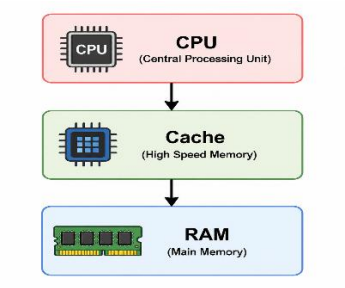
Problems



- Wrong instruction decoded.
- Incorrect memory address.
- Invalid arithmetic result.
- Data corruption.
- Program crash.
- Communication failure.

9. Efficient CPU Design to Reduce Errors

1. Cache Memory



- Faster data access.
- Reduces processing delay.

2. Instruction Pipelining

```
Instruction 1
Fetch → Decode → Execute

Instruction 2
     Fetch → Decode → Execute

Instruction 3
           Fetch → Decode → Execute
```

- Executes multiple instructions simultaneously.
- Improves throughput.

3. Error Detection

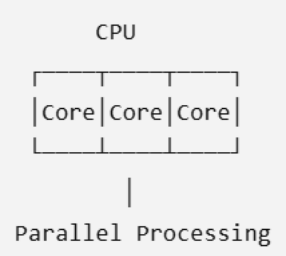
```
Packet
   |
Checksum
   |
Verify
   |
Correct / Error
```

- Detects corrupted packet data before execution.

4. Branch Prediction

- Predicts the next instruction.
- Reduces CPU waiting time.
- Improves performance.

5. Multi-Core Processor



- Processes multiple packets simultaneously.
- Improves system performance.

10. Benchmarking

Benchmarking is the process of measuring CPU performance using standard tests.

Importance

- Measures execution speed.
- Identifies processing errors.
- Compares CPU performance.
- Detects bottlenecks.
- Improves system reliability.

11. Real-Time Example

A network router receives a packet with the hexadecimal value 3A.

- Correct conversion:
  - $3A_{16} = 00111010_2$
- Incorrect conversion:
  - 00111110_2

The CPU interprets the wrong binary value, leading to incorrect packet processing. By using proper conversion methods, efficient CPU architecture, and benchmarking tools, such errors can be detected and corrected before affecting communication.

12. Advantages

- Accurate packet processing
- Faster communication
- Reduced processing errors
- Improved CPU performance
- Reliable instruction execution
- Better debugging
- High system efficiency

13. Conclusion

In data communication systems, hexadecimal values are used for easy representation, while the CPU executes instructions in binary. Accurate hexadecimal-to-binary conversion is essential to avoid incorrect instruction execution and data processing errors. Efficient CPU features such as cache memory, pipelining, branch prediction, multi-core processing, and benchmarking help improve performance, detect errors early, and ensure reliable real-time communication.

MARKS ALLOCATION

Criteria	Understanding of Problem Context (20%)	Application of Concepts (20%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (20%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand